

Machine Learning Foundations — Unsupervised Learning & Dimensionality Reduction

1. Supervised vs Unsupervised Learning

Supervised Learning

In supervised learning, the dataset contains pairs:

$$(x_i, y_i)$$

where:

x_i = input/features

y_i = target/label

Main tasks:

Regression → predict continuous values

Classification → predict categories/classes

The goals and evaluation methods are generally well-defined.

Unsupervised Learning

In unsupervised learning, we only have:

$$x_1, x_2, \dots, x_n$$

There are no target labels y .

Each x_i is a D -dimensional vector:

$$x_i \in \mathbb{R}^D$$

The broad goal is to understand the structure of the data.

Unsupervised learning can be viewed as a way to:

Compress data

Explain data

Group data

The two main tasks discussed are:

Dimensionality Reduction

Density Estimation

2. Why Unsupervised Learning Is Usually Not the Final Goal

Unsupervised learning is often used as a pre-processing step rather than an end in itself.

The output of an unsupervised algorithm may not have useful meaning automatically.

Humans often need to interpret the discovered structure.

Example: Coca-Cola Tweets

Suppose Coca-Cola receives 1 million tweets in a week.

It would be impossible for a marketing manager to read and analyze every tweet individually.

An unsupervised learning algorithm could group the tweets into, say:

1 million tweets



Clustering



10 groups

Possible groups might include:

- People posting selfies with Coke
- Businesses collaborating with Coke
- Paid promotions
- Customers buying Coke
- Complaints
- Product discussions

The algorithm creates the groups, but the human interprets what those groups mean.

Key idea

Unsupervised learning discovers structure; humans or downstream ML tasks often give that structure meaning.

3. Dimensionality Reduction

The first major unsupervised learning task discussed is:

Dimensionality Reduction

Main goal

Compression and simplification of data.

Suppose each data point has a very large number of features.

For example:

1,000,000 people

×

1,000,000 gene expression values

This produces an extremely large dataset.

Storing or transmitting such data directly can be impractical.

Dimensionality reduction attempts to represent the same information using fewer dimensions.

For example:

Original:

$D = 1,000,000$ dimensions

Reduced:

$D' = 100$ dimensions

The objective is to retain as much useful information as possible while reducing dimensionality.

4. Encoder and Decoder

Unlike many supervised learning problems where we learn one model, dimensionality reduction involves two functions:

Encoder

Decoder

Let:

$$x_i \in \mathbb{R}^D$$

Encoder

The encoder is represented by:

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^{D'}$$

where:

$$D' < D$$

The encoder compresses the original vector.

D-dimensional vector



Encoder



D'-dimensional vector

Decoder

The decoder is represented by:

$$g: \mathbb{R}^{D'} \rightarrow \mathbb{R}^D$$

It attempts to reconstruct the original data.

D'-dimensional vector



Decoder



D-dimensional vector

5. Overall Dimensionality Reduction Process

The complete process is:

Original data

x



Encoder f



Compressed representation

$f(x)$



Decoder g



Reconstructed data

$g(f(x))$

The ideal situation would be:

$$g(f(x_i)) = x_i$$

However, because we are compressing the data, exact reconstruction is generally impossible.

Therefore, we want:

$$g(f(x_i)) \approx x_i$$

6. Reconstruction Error

We need a way to measure how close the reconstructed data is to the original data. The lecture uses squared Euclidean distance:

$$\|g(f(x_i)) - x_i\|^2$$

A good encoder-decoder pair should make this value as small as possible.

For all n training examples, the average reconstruction error is:

$$\frac{1}{n} \sum_{i=1}^n \|g(f(x_i)) - x_i\|^2$$

Objective

Find f and g that minimize:

$$\frac{1}{n} \sum_{i=1}^n \|g(f(x_i)) - x_i\|^2$$

This is the central mathematical idea behind the dimensionality reduction discussion.

7. Simple Example

Suppose:

$$D = 2$$

and:

$$D' = 1$$

So we're trying to compress a 2D vector into a single number.

Our training data is:

$$x_1 = (1, 0.8)$$

$$x_2 = (2, 2.2)$$

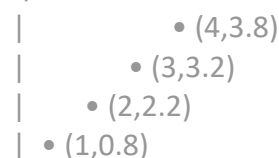
$$x_3 = (3, 3.2)$$

$$x_4 = (4, 3.8)$$

Visualized approximately:

x2

↑



+-----→ x1

The points approximately follow a diagonal line.

This is important because although the data technically has 2 dimensions, most of its variation appears to lie along approximately one dimension.

That's exactly the kind of situation where dimensionality reduction can work well.

8. First Encoder-Decoder Pair

Consider:

$$f(x) = x_1 - x_2$$

and:

$$g(u) = (u, u)$$

Encoder

For the four points:

$$f(x_1) = 1 - 0.8 = 0.2$$

$$f(x_2) = 2 - 2.2 = -0.2$$

$$f(x_3) = 3 - 3.2 = -0.2$$

$$f(x_4) = 4 - 3.8 = 0.2$$

So:

Original 2D data		Encoded 1D data
(1, 0.8)	→	0.2
(2, 2.2)	→	-0.2
(3, 3.2)	→	-0.2
(4, 3.8)	→	0.2

Notice something problematic:

$$(1, 0.8) \rightarrow 0.2$$

$$(4, 3.8) \rightarrow 0.2$$

Two very different points have been compressed into the same value.

9. Decoding the First Representation

The decoder is:

$$g(u) = (u, u)$$

Therefore:

$$g(0.2) = (0.2, 0.2)$$

and:

$$g(-0.2) = (-0.2, -0.2)$$

The reconstructed points become approximately:

Original		Reconstructed
(1, 0.8)	→	(0.2, 0.2)
(2, 2.2)	→	(-0.2, -0.2)
(3, 3.2)	→	(-0.2, -0.2)
(4, 3.8)	→	(0.2, 0.2)

This is a poor encoder-decoder pair.

Why?

Because the reconstructed points are far away from the original points.

The reconstruction error is large.

10. Second Encoder-Decoder Pair

Now consider:

$$\tilde{f}(x) = \frac{x_1 + x_2}{2}$$

and:

$$\tilde{g}(u) = (u, u)$$

This encoder calculates approximately the average of the two dimensions.

Encoding

For $x_1 = (1, 0.8)$:

$$\tilde{f}(x_1) = \frac{1 + 0.8}{2} = 0.9$$

For $x_2 = (2, 2.2)$:

$$\tilde{f}(x_2) = \frac{2 + 2.2}{2} = 2.1$$

For $x_3 = (3, 3.2)$:

$$\tilde{f}(x_3) = 3.1$$

For $x_4 = (4, 3.8)$:

$$\tilde{f}(x_4) = 3.9$$

So:

Original		Encoded
(1,0.8)	→	0.9
(2,2.2)	→	2.1
(3,3.2)	→	3.1
(4,3.8)	→	3.9

11. Decoding the Second Representation

Using:

$$\tilde{g}(u) = (u, u)$$

we get:

Encoded		Reconstructed
0.9	→	(0.9,0.9)
2.1	→	(2.1,2.1)
3.1	→	(3.1,3.1)

3.9 → (3.9, 3.9)

Compare these with the original points:

Original Reconstructed

(1, 0.8) (0.9, 0.9)

(2, 2.2) (2.1, 2.1)

(3, 3.2) (3.1, 3.1)

(4, 3.8) (3.9, 3.9)

The reconstructed points are very close to the originals.
Therefore:

$$\tilde{g}(\tilde{f}(x_i)) \approx x_i$$

12. Comparing the Two Encoder-Decoder Pairs

First pair

$$f(x) = x_1 - x_2$$

$$g(u) = (u, u)$$

Produces poor reconstruction.

Second pair

$$\tilde{f}(x) = \frac{x_1 + x_2}{2}$$

$$\tilde{g}(u) = (u, u)$$

Produces much better reconstruction.

Therefore:

$$(\tilde{f}, \tilde{g})$$

is a better encoder-decoder pair than:

$$(f, g)$$

because it gives a smaller reconstruction error.

13. What Is Really Happening?

Originally:

2 dimensions



[x_1 , x_2]

After dimensionality reduction:

2 dimensions



1 dimension



[u]

Then reconstruction:

$[u]$

$\downarrow$

$[u, u]$

We are storing fewer numbers, but trying to preserve as much information as possible.
This is the fundamental idea of dimensionality reduction.

14. Real Dimensionality Reduction

The example above is deliberately simplified.

In the example, we only considered two possible encoder-decoder pairs:

(f, g)

or

$(\tilde{f}, \tilde{g})$

The algorithm simply chooses the better pair.

In a real dimensionality reduction algorithm, there can be a very large/infinite number of possible functions.

The algorithm attempts to learn:

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^{D'}$$

and:

$$g: \mathbb{R}^{D'} \rightarrow \mathbb{R}^D$$

that minimize reconstruction error.

15. Important Mathematical Summary

Input

$$x_i \in \mathbb{R}^D$$

Encoder

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^{D'}$$

where:

$$D' < D$$

Decoder

$$g: \mathbb{R}^{D'} \rightarrow \mathbb{R}^D$$

Reconstruction

$$\hat{x}_i = g(f(x_i))$$

Goal

$$\hat{x}_i \approx x_i$$

Reconstruction loss

$$\frac{1}{n} \sum_{i=1}^n \|g(f(x_i)) - x_i\|^2$$

Minimize this loss to find a good encoder and decoder.

16. Key Terms to Remember

Term	Meaning
Unsupervised Learning	Learning from data without target labels
Dimensionality Reduction	Representing data using fewer dimensions
Encoder	Converts high-dimensional data into a lower-dimensional representation
Decoder	Reconstructs the original-dimensional data
Latent/Compressed Representation	Lower-dimensional representation produced by the encoder
Reconstruction	Output obtained after decoding
Reconstruction Error	Difference between original and reconstructed data
Compression	Representing data using fewer values
Density Estimation	Another major unsupervised learning task

17. Exam/Interview Points ★

Q: What is unsupervised learning?

Learning from data where the examples have no target labels y .

Q: What are the two main tasks discussed?

Dimensionality reduction

Density estimation

Q: What is the goal of dimensionality reduction?

Compression and simplification while preserving as much information as possible.

Q: What are the two models in dimensionality reduction?

Encoder f

Decoder g

Q: What does the encoder do?

$$f: \mathbb{R}^D \rightarrow \mathbb{R}^{D'}$$

It reduces the dimensionality.

Q: What does the decoder do?

$$g: \mathbb{R}^{D'} \rightarrow \mathbb{R}^D$$

It reconstructs the original-dimensional data.

Q: What are we trying to achieve?

$$g(f(x_i)) \approx x_i$$

Q: How do we measure reconstruction quality?

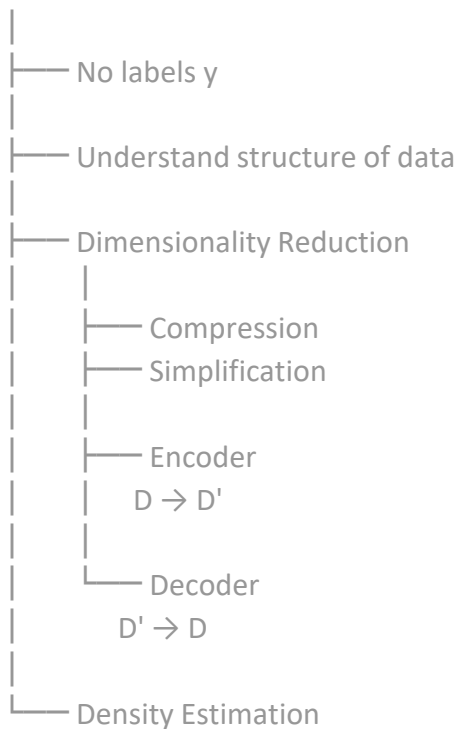
Using squared reconstruction error:

$$\|g(f(x_i)) - x_i\|^2$$

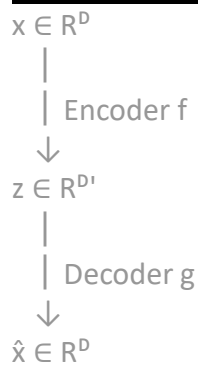
and average it across the training examples.

18. Quick Revision Sheet

UNSUPERVISED LEARNING



Dimensionality Reduction



Goal:

$$\hat{x} = g(f(x)) \approx x$$

Loss:

$$\frac{1}{n} \sum_i \|g(f(x_i)) - x_i\|^2$$

Core idea: Reduce the number of dimensions while keeping the reconstructed data as close as possible to the original data.

From <<https://chatgpt.com/uc/6ab9c0fb-04e8-83ea-8f8c-7f82bea78baa>>